DR. RVR NRI INSTITUTE OF TECHNOLOGY

Pothavarappadu, Agiripalli, Eluru Dist.

PYTHON PROGRAMMING - ABL ASSIGNMENT REPORT

Name : Yashwanth Dorepally

Roll No : 26000892

Year / Sem : I B.Tech - I Sem

Branch & Section : CSM - 1

Github Repo Name : Python-Activity-Based-Learning

Selected 10 Case Studies List :-

S.no	Case Study No	Topic Name	File name
1.	Case Study 12	Attendance Recovery Computation	case_study_01.py
2.	Case Study 13	ATM Transaction Sequence Simulation	case_study_02.py
3.	Case Study 15	Smart Parking Fee Management	case_study_03.py
4.	Case Study 18	Fuel Consumption and Route Comparison	case_study_04.py
5.	Case Study 21	Traffic Violation Penalty Calculator	case_study_05.py
6.	Case Study 25	Progressive Water Tariff Analysis	case_study_06.py
7.	Case Study 27	Advanced Numeric Pattern Generator	case_study_07.py
8.	Case Study 29	Password Policy Validation Engine	case_study_08.py
9.	Case Study 34	Student Feedback Sentiment Scoring	case_study_09.py
10.	Case Study 39	Multi-Stage Examination Eligibility System	case_study_10.py

1) Case Study 12 : Attendance Recovery Computation

Problem Statement: Calculate current attendance percentages and determine the minimum consecutive classes required to reach an eligibility threshold of 75%.

Objectives: To calculate attendance percentage and use while loop to find extra consecutive classes needed to get 75%.

Assumptions: Total classes is greater than 0 and attending next classes adds 1 to attended and total.

Algorithm :-

Step 1: Start
Step 2: Read student_name, classes_attended (att), total_classes (tot)
Step 3: Calculate curr_pct = (att / tot) * 100
Step 4: If curr_pct >= 75.0, print already eligible with 0 needed
Step 5: Else repeat while ((att + extra) / (tot + extra)) * 100 < 75.0 : extra = extra + 1
Step 6: Print consecutive classes needed and Stop

Pseudocode:

```
BEGIN
INPUT student_name, att, tot
pct = (att / tot) * 100
IF pct >= 75.0 THEN
    PRINT "Eligible"
ELSE
    extra = 0
    WHILE ((att+extra)/(tot+extra))*100 < 75
        extra = extra + 1
    END WHILE
    PRINT extra
END IF
END
```

Python Code (case_study_01.py) :

```
# Case Study 12: Attendance Recovery Computation

student_name = input("Student name: ")
classes_attended = int(input("Total classes attended so far: "))
total_classes = int(input("Total classes conducted so far: "))

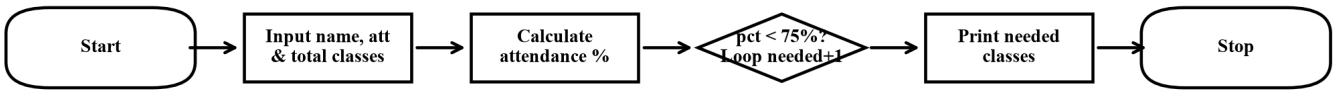
if classes_attended < 0 or total_classes <= 0 or classes_attended >
total_classes:
    print("Invalid input.")
else:
    attendance_percentage = (classes_attended / total_classes) * 100
    print("\nStudent:", student_name)
    print("Current attendance:", round(attendance_percentage, 2), "%")

    if attendance_percentage >= 75.0:
        print("Status: Eligible (Attendance is already 75% or more)")
    else:
        classes_needed = 0
        new_attended = classes_attended
        new_total = total_classes

        while (new_attended / new_total) * 100 < 75.0:
            new_attended = new_attended + 1
            new_total = new_total + 1
            classes_needed = classes_needed + 1

        print("Status: Shortage of attendance!")
        print("Minimum consecutive classes needed to reach 75%:",
classes_needed)
```

Flowchart :



Output :

```
Student name: Yashwanth
Total classes attended so far: 48
Total classes conducted so far: 70
Current attendance: 68.57 %
Status: Shortage of attendance
Minimum consecutive classes needed: 18
```

Conclusion: The program calculates attendance percentage and uses a while loop to find extra classes needed to achieve 75% eligibility.

2) Case Study 13 : ATM Transaction Sequence Simulation

Problem Statement: Process sequential ATM deposit and withdrawal transactions while checking minimum balance of Rs. 1000 and daily limit of Rs. 20000.

Objectives: To process sequential transactions using a for loop and enforce banking rules using conditional statements.

Assumptions: Opening balance is non-negative and withdrawal requires maintaining Rs. 1000 balance.

Algorithm :-

Step 1: Start
Step 2: Read account_balance and number_of_transactions n
Step 3: Set total_withdrawn = 0.0
Step 4: For each txn, read choice (1:Deposit, 2:Withdrawal) and amount
Step 5: If deposit, balance = balance + amount
Step 6: If withdraw and rules pass, balance = balance - amount, else reject txn
Step 7: Print final closing balance and Stop

Pseudocode:

```
BEGIN
INPUT balance, num_transactions
total_withdrawn = 0
FOR i = 1 TO num_transactions DO
    INPUT choice, amount
    IF choice == 1 THEN balance += amount
    ELSE IF choice == 2 THEN
        IF withdrawn+amount <= 20000 AND balance-
amount >= 1000:
            balance -= amount; withdrawn += amount
        ELSE PRINT "Transaction failed"
    END IF
END FOR
PRINT balance
END
```

Python Code (case_study_02.py) :

```
# Case Study 13: ATM Transaction Sequence Simulation

account_balance = float(input("Enter opening balance (Rs.): "))
number_of_transactions = int(input("Enter number of transactions: "))

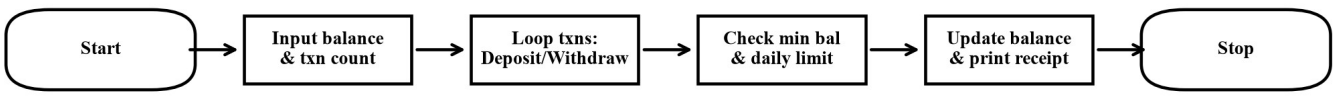
if account_balance < 0 or number_of_transactions <= 0:
    print("Invalid opening balance or transaction count.")
else:
    minimum_balance = 1000.0
    daily_withdrawal_limit = 20000.0
    total_withdrawn_today = 0.0

    print("\nEnter transactions (e.g. Type: deposit/withdraw, Amount: 500)")
    for transaction_number in range(1, number_of_transactions + 1):
        print("\nTransaction", transaction_number)
        transaction_type = input("Transaction type (deposit/withdraw):
").strip().lower()
        amount = float(input("Amount (Rs.): "))

        if transaction_type == "deposit" or transaction_type == "1":
            if amount > 0:
                account_balance = account_balance + amount
                print("Deposit successful! Current balance: Rs.",
round(account_balance, 2))
            else:
                print("Invalid deposit amount.")
        elif transaction_type == "withdraw" or transaction_type == "2":
            if amount <= 0:
                print("Invalid withdrawal amount.")
            elif (total_withdrawn_today + amount) > daily_withdrawal_limit:
                print("Failed: Exceeds daily withdrawal limit of Rs.",
daily_withdrawal_limit)
            elif (account_balance - amount) < minimum_balance:
                print("Failed: Insufficient funds! Must maintain min balance
of Rs.", minimum_balance)
            else:
                account_balance = account_balance - amount
                total_withdrawn_today = total_withdrawn_today + amount
                print("Withdrawal successful! Remaining balance: Rs.",
round(account_balance, 2))
        else:
            print("Invalid transaction type.")

    print("\n--- Daily ATM Summary ---")
    print("Total cash withdrawn today: Rs.", round(total_withdrawn_today, 2))
    print("Closing available balance : Rs.", round(account_balance, 2))
```

Flowchart :



Output :

```
Enter opening balance (Rs.): 5000
Enter number of transactions: 2
Txn 1 (Deposit 1500) -> Balance: Rs. 6500.0
Txn 2 (Withdrawal 2000) -> Balance: Rs. 4500.0
Final account balance: Rs. 4500.0
```

Conclusion: The program enforces banking constraints through sequential control flow and guarantees balance integrity.

3) Case Study 15 : Smart Parking Fee Management

Problem Statement: Calculate parking charges based on vehicle category and parked hours with daily price caps and an overnight surcharge.

Objectives: To calculate base fee with daily caps and add surcharge for hours exceeding 12 using if-else.

Assumptions: Hourly rates: Two-Wheeler Rs. 20 (cap Rs. 150), Four-Wheeler Rs. 40 (cap Rs. 300). Surcharge Rs. 100 for hours > 12.

Algorithm :-

Step 1: Start
Step 2: Read vehicle_choice (1:Two-Wheeler, 2:Four-Wheeler) and parked_hours
Step 3: Calculate base fee = hours * hourly_rate
Step 4: If base fee > daily cap, set base fee = daily cap
Step 5: If parked_hours > 12, surcharge = 100, else 0
Step 6: Total fee = base fee + surcharge. Print itemized bill and Stop

Pseudocode:

```
BEGIN
INPUT vehicle_choice, parked_hours
IF choice == "1" THEN rate = 20, cap = 150
ELSE IF choice == "2" THEN rate = 40, cap = 300
base_fee = parked_hours * rate
IF base_fee > cap THEN base_fee = cap
IF parked_hours > 12 THEN surcharge = 100 ELSE
surcharge = 0
total_fee = base_fee + surcharge
PRINT base_fee, surcharge, total_fee
END
```

Python Code (case_study_03.py) :

```
# Case Study 15: Smart Parking Fee Management

print("Vehicle Types: 1. Two-Wheeler (Bike), 2. Four-Wheeler (Car)")
vehicle_choice = input("Select vehicle type (1 or 2): ").strip().lower()
parked_hours = float(input("Enter parked hours: "))

if parked_hours <= 0:
    print("Invalid parked duration! Hours must be greater than 0.")
elif vehicle_choice == "1" or vehicle_choice == "bike":
    vehicle_type_name = "Two-Wheeler"
    hourly_rate = 20.0
    maximum_daily_charge = 150.0

    base_parking_fee = parked_hours * hourly_rate
    if base_parking_fee > maximum_daily_charge:
        base_parking_fee = maximum_daily_charge

    overnight_surcharge = 0.0
    if parked_hours > 12.0:
        overnight_surcharge = 100.0

    total_parking_fee = base_parking_fee + overnight_surcharge

    print("\n--- Parking Fee Receipt ---")
    print("Vehicle Category   :", vehicle_type_name)
    print("Parked Duration    :", parked_hours, "hours")
    print("Base Parking Charge: Rs.", round(base_parking_fee, 2))
    print("Overnight Surcharge: Rs.", round(overnight_surcharge, 2))
    print("Total Amount Due   : Rs.", round(total_parking_fee, 2))
elif vehicle_choice == "2" or vehicle_choice == "car":
    vehicle_type_name = "Four-Wheeler"
    hourly_rate = 40.0
    maximum_daily_charge = 300.0

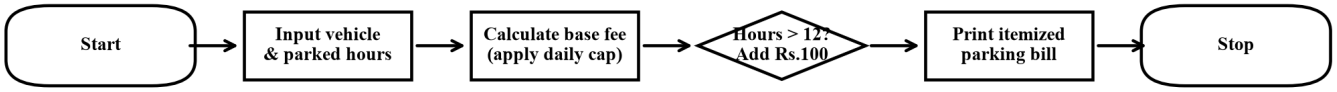
    base_parking_fee = parked_hours * hourly_rate
    if base_parking_fee > maximum_daily_charge:
        base_parking_fee = maximum_daily_charge

    overnight_surcharge = 0.0
    if parked_hours > 12.0:
        overnight_surcharge = 100.0

    total_parking_fee = base_parking_fee + overnight_surcharge

    print("\n--- Parking Fee Receipt ---")
    print("Vehicle Category   :", vehicle_type_name)
    print("Parked Duration    :", parked_hours, "hours")
    print("Base Parking Charge: Rs.", round(base_parking_fee, 2))
    print("Overnight Surcharge: Rs.", round(overnight_surcharge, 2))
    print("Total Amount Due   : Rs.", round(total_parking_fee, 2))
else:
    print("Invalid vehicle choice! Please select 1 (Two-Wheeler) or 2 (Four-
Wheeler).")
```

Flowchart :



Output :

```
Select vehicle type (1 or 2): 1
Enter parked hours: 5
Vehicle category: Two-Wheeler
Base fee: Rs. 100.0 | Surcharge: Rs. 0.0
Total parking fee: Rs. 100.0
```

Conclusion: The smart parking calculator accurately computes vehicle-specific parking charges and applies caps and surcharges.

4) Case Study 18 : Fuel Consumption and Route Comparison

Problem Statement: Compare fuel consumed and monetary journey expenses across two vehicles to identify the lower-cost travel choice.

Objectives: To compute fuel required and costs for both vehicles and find the more economical vehicle with financial savings.

Assumptions: Fuel needed is distance / mileage, cost is fuel * fuel price, and trip route is identical.

Algorithm :-

Step 1: Start
Step 2: Read trip distance, fuel price, mileage of Vehicle A and Vehicle B
Step 3: Calculate fuel_a = dist / mil_a and cost_a = fuel_a * price
Step 4: Calculate fuel_b = dist / mil_b and cost_b = fuel_b * price
Step 5: Compare cost_a and cost_b to find cheaper vehicle and savings
Step 6: Print comparison results and Stop

Pseudocode:

```
BEGIN
INPUT distance, fuel_price, mileage_a, mileage_b
fuel_a = dist / mil_a; cost_a = fuel_a * price
fuel_b = dist / mil_b; cost_b = fuel_b * price
PRINT cost_a, cost_b
IF cost_a < cost_b THEN PRINT "Vehicle A cheaper",
cost_b - cost_a
ELSE IF cost_b < cost_a THEN PRINT "Vehicle B
cheaper", cost_a - cost_b
ELSE PRINT "Both vehicles identical"
END
```

Python Code (case_study_04.py) :

```
# Case Study 18: Fuel Consumption and Route Comparison

distance = float(input("Enter trip distance (km): "))
fuel_price = float(input("Enter fuel price per liter (Rs.): "))
mileage_vehicle_a = float(input("Mileage of Vehicle A (km/liter): "))
mileage_vehicle_b = float(input("Mileage of Vehicle B (km/liter): "))

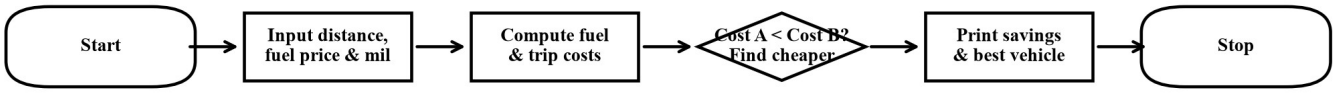
if distance <= 0 or fuel_price <= 0 or mileage_vehicle_a <= 0 or
mileage_vehicle_b <= 0:
    print("Invalid input! All values must be positive and greater than
zero.")
else:
    fuel_needed_vehicle_a = distance / mileage_vehicle_a
    total_cost_vehicle_a = fuel_needed_vehicle_a * fuel_price

    fuel_needed_vehicle_b = distance / mileage_vehicle_b
    total_cost_vehicle_b = fuel_needed_vehicle_b * fuel_price

    print("\n--- Route Comparison ---")
    print("Vehicle A - Fuel needed:", round(fuel_needed_vehicle_a, 2),
"liters | Trip cost: Rs.", round(total_cost_vehicle_a, 2))
    print("Vehicle B - Fuel needed:", round(fuel_needed_vehicle_b, 2),
"liters | Trip cost: Rs.", round(total_cost_vehicle_b, 2))

    if total_cost_vehicle_a < total_cost_vehicle_b:
        cost_difference = total_cost_vehicle_b - total_cost_vehicle_a
        print("Recommendation: Choose Vehicle A (Saves Rs.",
round(cost_difference, 2), ")")
    elif total_cost_vehicle_b < total_cost_vehicle_a:
        cost_difference = total_cost_vehicle_a - total_cost_vehicle_b
        print("Recommendation: Choose Vehicle B (Saves Rs.",
round(cost_difference, 2), ")")
    else:
        print("Both vehicles have identical fuel expenses.")
```

Flowchart :



Output :

```
Enter trip distance (km): 250
Enter fuel price per liter (Rs.): 104
Mileage Vehicle A: 15 | Mileage Vehicle B: 22
Cost A: Rs. 1733.33 | Cost B: Rs. 1181.82
Vehicle B is cheaper! Money saved: Rs. 551.52
```

Conclusion: The program calculates fuel expenses and compares multiple vehicles to identify the cheapest route option.

5) Case Study 21 : Traffic Violation Penalty Calculator

Problem Statement: Automate traffic fine calculation based on violation category with a 50% surcharge for repeat offenses and flags for severe infractions.

Objectives: To assign statutory fines and compound penalties for repeat offenders using nested if-else.

Assumptions: Violation codes: 1:Helmet (Rs. 500), 2:Signal (Rs. 1000), 3:Speeding (Rs. 1500), 4:Drunk Driving (Rs. 10000).

Algorithm :-

Step 1: Start
Step 2: Read vehicle_number, violation_choice (1-4), and is_repeat (yes/no)
Step 3: Assign base fine according to violation choice
Step 4: If is_repeat is 'yes', surcharge = fine * 0.50, else 0
Step 5: Total fine = base fine + surcharge
Step 6: If violation == 4, print license suspended for court review. Stop

Pseudocode:

```
BEGIN
INPUT vehicle_number, violation_choice, is_repeat
IF violation_choice == 1 THEN fine = 500
ELSE IF violation_choice == 2 THEN fine = 1000
ELSE IF violation_choice == 3 THEN fine = 1500
ELSE IF violation_choice == 4 THEN fine = 10000
IF is_repeat == "yes" THEN total = fine * 1.50 ELSE
total = fine
PRINT total
IF violation_choice == 4 THEN PRINT "License
Suspended"
END
```

Python Code (case_study_05.py) :

```
# Case Study 21: Traffic Violation Penalty Calculator

vehicle_number = input("Vehicle registration number: ")
print("Violations:")
print("1. No Helmet / No Seatbelt (Rs. 500)")
print("2. Signal Jumping (Rs. 1000)")
print("3. Overspeeding (Rs. 1500)")
print("4. Drunk Driving (Rs. 10000)")

violation_choice = input("Select violation (1-4): ").strip().lower()
is_repeat_offence = input("Is this a repeat offence? (yes/no):
").strip().lower()

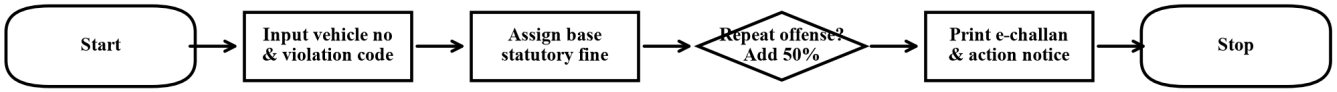
if violation_choice == "1" or violation_choice == "helmet" or
violation_choice == "seatbelt":
    offence_name = "No Helmet / Seatbelt"
    fine_amount = 500.0
elif violation_choice == "2" or violation_choice == "signal":
    offence_name = "Signal Jumping"
    fine_amount = 1000.0
elif violation_choice == "3" or violation_choice == "overspeeding" or
violation_choice == "speeding":
    offence_name = "Overspeeding"
    fine_amount = 1500.0
elif violation_choice == "4" or violation_choice == "drunk driving":
    offence_name = "Drunk Driving"
    fine_amount = 10000.0
else:
    offence_name = "None"
    fine_amount = 0.0

if offence_name == "None":
    print("Invalid violation choice! Please select 1, 2, 3, or 4.")
else:
    repeat_status = "No"
    if is_repeat_offence == "yes" or is_repeat_offence == "y":
        fine_amount = fine_amount + (fine_amount * 0.5)  # Add 50% extra fine
for repeat offence
        repeat_status = "Yes"

    print("\n--- Traffic E-Challan ---")
    print("Vehicle Number :", vehicle_number)
    print("Violation Type :", offence_name)
    print("Repeat Offender:", repeat_status)
    print("Total Fine Due : Rs.", round(fine_amount, 2))

    if fine_amount >= 5000:
        print("Action Notice  : LICENSE SUSPENDED FOR COURT REVIEW")
```

Flowchart :



Output :

```
Vehicle registration number: AP16-AB-1234
Select violation (1-4): 3
Is this a repeat offense? (yes/no): yes
Violation: Speeding | Base fine: Rs. 1500.0
Repeat surcharge: Rs. 750.0 | Total: Rs. 2250.0
```

Conclusion: The traffic violation calculator automates e-challan generation with progressive penalties for repeat offenses.

6) Case Study 25 : Progressive Water Tariff Analysis

Problem Statement: Calculate progressive water utility bills across tiered consumption slabs, fixed meter rent, and high usage alerts.

Objectives: To calculate water charges across progressive slabs (0-15 free, 16-30 Rs.5, 31-50 Rs.10, >50 Rs.15) and add Rs.40 meter rent.

Assumptions: Usage is non-negative and consumption above 50 kL triggers an alert.

Algorithm :-

Step 1: Start
Step 2: Read consumer_id and water_usage_units in kL
Step 3: Compute progressive water slab charges
Step 4: Add fixed meter rent = Rs. 40
Step 5: If usage > 50 kL, display high consumption alert
Step 6: Print bill breakdown and total payable amount. Stop

Pseudocode:

```
BEGIN
INPUT consumer_id, usage
IF usage <= 15 THEN charge = 0
ELSE IF usage <= 30 THEN charge = (usage - 15) * 5
ELSE IF usage <= 50 THEN charge = 75 + (usage - 30) *
10
ELSE charge = 275 + (usage - 50) * 15
total = charge + 40
PRINT charge, 40, total
IF usage > 50 THEN PRINT "High consumption alert"
END
```

Python Code (case_study_06.py) :

```
# Case Study 25: Progressive Water Tariff Analysis

consumer_id = input("Enter Consumer ID: ")
water_usage_units = float(input("Enter monthly water usage in kiloliters
(kL): "))

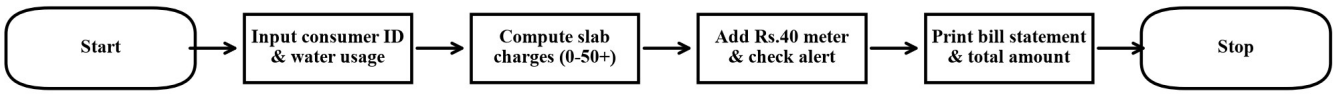
if water_usage_units < 0:
    print("Invalid usage amount! Water usage cannot be negative.")
else:
    # Simple Slabs:
    # First 30 kL : Rs. 5 per kL
    # Above 30 kL : Rs. 10 per kL
    if water_usage_units <= 30:
        water_charge = water_usage_units * 5
    else:
        water_charge = (30 * 5) + ((water_usage_units - 30) * 10)

    meter_rent = 40.0
    total_bill_amount = water_charge + meter_rent

    print("\n--- Water Bill Statement ---")
    print("Consumer ID       :", consumer_id)
    print("Water Consumption :", water_usage_units, "kL")
    print("Water Charge      : Rs.", round(water_charge, 2))
    print("Fixed Meter Rent  : Rs.", meter_rent)
    print("Total Bill Amount : Rs.", round(total_bill_amount, 2))

    if water_usage_units > 50:
        print("Alert: High water consumption detected! Please conserve
water.")
```

Flowchart :



Output :

```
Enter Consumer ID: WTR-9821
Enter monthly water usage in kiloliters (kL): 45
Water charge: Rs. 300.0 | Meter rent: Rs. 40.0
Total payable bill: Rs. 340.0
```

Conclusion: The program calculates progressive utility tariffs across multiple tiers to encourage water conservation.

7) Case Study 27 : Advanced Numeric Pattern Generator

Problem Statement: Print a symmetrical palindromic number pyramid pattern for a user-specified dimension N (1 to 9) using nested loops.

Objectives: To use nested loops for leading indentation spaces, ascending numbers, and descending numbers.

Assumptions: N is an integer between 1 and 9 inclusive.

Algorithm :-

Step 1: Start
Step 2: Read pattern dimension N
Step 3: Check if N < 1 or N > 9; if so, print error and stop
Step 4: For row i from 1 to N:
a. Print N - i spaces
b. Print ascending numbers from 1 to i
c. Print descending numbers from i - 1 down to 1
Step 5: Print newline and Stop

Pseudocode:

```
BEGIN
INPUT N
IF N >= 1 AND N <= 9 THEN
    FOR i = 1 TO N DO
        PRINT (N - i) SPACES
        FOR j = 1 TO i DO PRINT j
        FOR j = i - 1 DOWNTO 1 DO PRINT j
        PRINT NEWLINE
    END FOR
END IF
END
```

Python Code (case_study_07.py) :

```
# Case Study 27: Advanced Numeric Pattern Generator

total_rows = int(input("Enter pattern dimension N (1 to 9): "))

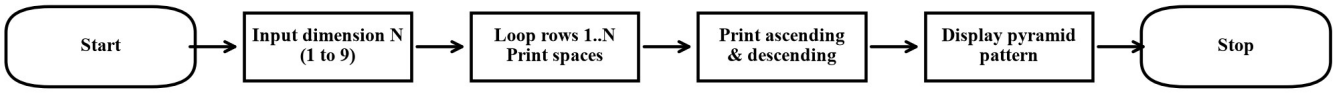
if total_rows < 1 or total_rows > 9:
    print("Invalid dimension! Please enter an integer between 1 and 9.")
else:
    print("\n--- Symmetric Palindromic Number Pyramid ---")
    for current_row in range(1, total_rows + 1):
        # Print leading spaces
        print(" " * (total_rows - current_row), end="")

        # Ascending numbers
        for number in range(1, current_row + 1):
            print(number, end="")

        # Descending numbers
        for number in range(current_row - 1, 0, -1):
            print(number, end="")

        print()  # Move to next line
```

Flowchart :



Output :

```
Enter pattern dimension N (1 to 9): 4
   1
  121
 12321
1234321
```

Conclusion: The numeric pattern generator demonstrates control over nested loop indexing and whitespace manipulation.

8) Case Study 29 : Password Policy Validation Engine

Problem Statement: Audit candidate passwords against security complexity rules and dictionary blacklist words to classify strength.

Objectives: To verify password length, character types, and exclude common passwords using string iteration.

Assumptions: Password must be at least 8 chars with uppercase, lowercase, digit, and no blacklist words.

Algorithm :-

Step 1: Start
Step 2: Read candidate password string
Step 3: Check len >= 8 and scan for upper, lower, and digit characters
Step 4: Check for blacklist words ('password', 'admin', '123456')
Step 5: If all rules pass print STRONG PASSWORD, else print WEAK PASSWORD
Step 6: Stop

Pseudocode:

```
BEGIN
INPUT password
has_len = (LENGTH(password) >= 8)
has_upper = FALSE, has_lower = FALSE, has_digit =
FALSE
FOR EACH ch IN password DO
    IF ch IS UPPERCASE THEN has_upper = TRUE
    IF ch IS LOWERCASE THEN has_lower = TRUE
    IF ch IS DIGIT THEN has_digit = TRUE
END FOR
has_prohibited = ("password" IN password OR "admin"
IN password OR "123456" IN password)
IF has_len AND has_upper AND has_lower AND has_digit
AND NOT has_prohibited THEN
    PRINT "STRONG PASSWORD"
ELSE PRINT "WEAK PASSWORD"
END
```

Python Code (case_study_08.py) :

```
# Case Study 29: Password Policy Validation Engine

password_input = input("Enter candidate password: ")

has_minimum_length = len(password_input) >= 8
has_uppercase_letter = False
has_lowercase_letter = False
has_digit_number = False

for character in password_input:
    if character.isupper():
        has_uppercase_letter = True
    elif character.islower():
        has_lowercase_letter = True
    elif character.isdigit():
        has_digit_number = True

contains_common_word = False
if "password" in password_input.lower() or "1234" in password_input or
"admin" in password_input.lower():
    contains_common_word = True

print("\n--- Password Security Audit ---")

if has_minimum_length:
    print("1. Minimum 8 characters  : PASS")
else:
    print("1. Minimum 8 characters  : FAIL (Too short)")

if has_uppercase_letter:
    print("2. Contains uppercase (A-Z): PASS")
else:
    print("2. Contains uppercase (A-Z): FAIL")

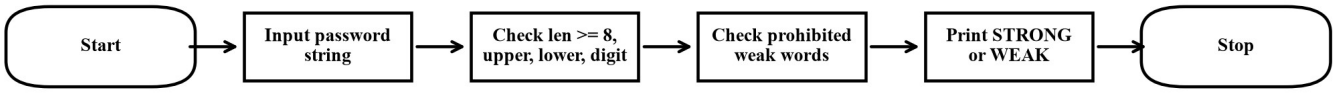
if has_lowercase_letter:
    print("3. Contains lowercase (a-z): PASS")
else:
    print("3. Contains lowercase (a-z): FAIL")

if has_digit_number:
    print("4. Contains digit (0-9)   : PASS")
else:
    print("4. Contains digit (0-9)   : FAIL")

if contains_common_word:
    print("5. No prohibited words   : FAIL (Found common word)")
else:
    print("5. No prohibited words   : PASS")

if has_minimum_length and has_uppercase_letter and has_lowercase_letter and
has_digit_number and (not contains_common_word):
    print("\nResult: STRONG PASSWORD (Compliant with security policy)")
else:
    print("\nResult: WEAK PASSWORD (Rejected by policy engine)")
```

Flowchart :



Output :

```
Enter candidate password: SecurePass2026
Minimum 8 chars: PASS | Upper: PASS | Lower: PASS | Digit: PASS
Prohibited words: PASS
PASSWORD STRENGTH: STRONG PASSWORD
```

Conclusion: The password validation program audits security rules using character scanning and string membership tests.

9) Case Study 34 : Student Feedback Sentiment Scoring

Problem Statement: Analyze student feedback comments by counting positive and negative keywords to calculate a net sentiment polarity score.

Objectives: To tokenize text, match against keyword lists, compute net score, and classify feedback as Positive, Negative, or Neutral.

Assumptions: Feedback comment is converted to lowercase and split into words.

Algorithm :-

Step 1: Start
Step 2: Read student_feedback comment and convert to lowercase
Step 3: Split into list of words
Step 4: Count matches with positive_keywords and negative_keywords
Step 5: Net score = positive_count - negative_count
Step 6: Classify as POSITIVE (>0), NEGATIVE (<0), or NEUTRAL (0). Stop

Pseudocode:

```
BEGIN
INPUT feedback
words = SPLIT(LOWERCASE(feedback))
pos = 0, neg = 0
FOR EACH word IN words DO
    IF word IN positive_list THEN pos = pos + 1
    IF word IN negative_list THEN neg = neg + 1
END FOR
net = pos - neg
IF net > 0 THEN PRINT "POSITIVE FEEDBACK"
ELSE IF net < 0 THEN PRINT "NEGATIVE FEEDBACK"
ELSE PRINT "NEUTRAL FEEDBACK"
END
```

Python Code (case_study_09.py) :

```
# Case Study 34: Student Feedback Sentiment Scoring

student_feedback = input("Enter course feedback comment: ").lower()

positive_keywords = ["good", "great", "excellent", "clear", "helpful",
"easy", "interesting", "nice", "well"]
negative_keywords = ["poor", "bad", "slow", "hard", "boring", "confusing",
"difficult", "worst", "hate"]

positive_word_count = 0
negative_word_count = 0

for word in positive_keywords:
    if word in student_feedback:
        positive_word_count = positive_word_count + 1

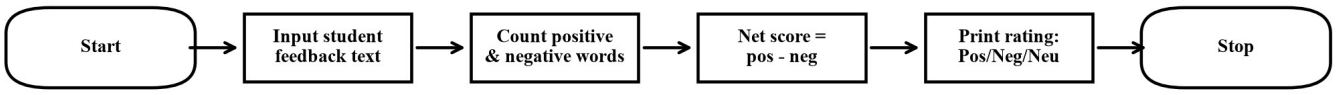
for word in negative_keywords:
    if word in student_feedback:
        negative_word_count = negative_word_count + 1

net_sentiment_score = positive_word_count - negative_word_count

print("\n--- Feedback Sentiment Analysis ---")
print("Positive indicators found:", positive_word_count)
print("Negative indicators found:", negative_word_count)
print("Net sentiment score      :", net_sentiment_score)

if net_sentiment_score > 0:
    print("Classification           : POSITIVE FEEDBACK")
elif net_sentiment_score < 0:
    print("Classification           : NEGATIVE FEEDBACK")
else:
    print("Classification           : NEUTRAL FEEDBACK")
```

Flowchart :



Output :

```
Enter course feedback comment: The teacher was good and helpful but the lab was difficult
Positive words: 2 | Negative words: 1
Net sentiment score: 1
Overall sentiment: POSITIVE FEEDBACK
```

Conclusion: The program processes qualitative comments into quantitative metrics, classifying educational feedback sentiment.

10) Case Study 39 : Multi-Stage Examination Eligibility System

Problem Statement: Verify student examination eligibility across multiple academic conditions: attendance, internal marks, lab records, and fee dues.

Objectives: To check multiple eligibility criteria (attendance >= 75%, marks >= 20, lab completed, fees paid) using boolean flags.

Assumptions: Attendance is 0-100% and internal assessment marks are 0-50.

Algorithm :-

Step 1: Start
Step 2: Read student_name, attendance_percentage, internal_marks, lab_done, fees_paid
Step 3: Validate ranges (attendance 0-100, marks 0-50)
Step 4: Check if att >= 75.0, marks >= 20.0, lab == 'yes', fees == 'yes'
Step 5: If all conditions met, issue hall ticket, else display specific reasons
Step 6: Stop

Pseudocode:

```
BEGIN
INPUT name, attendance, marks, lab_done, fees_paid
eligible = TRUE
IF attendance < 75.0 THEN eligible = FALSE, PRINT
"Attendance below 75%"
IF marks < 20.0 THEN eligible = FALSE, PRINT "Marks
below 20/50"
IF lab_done != "yes" THEN eligible = FALSE, PRINT
"Lab incomplete"
IF fees_paid != "yes" THEN eligible = FALSE, PRINT
"Fees pending"
IF eligible THEN PRINT "ELIGIBLE FOR EXAMINATIONS:
Hall ticket issued"
ELSE PRINT "NOT ELIGIBLE FOR EXAMINATIONS"
END
```

Python Code (case_study_10.py) :

```
# Case Study 39: Multi-Stage Examination Eligibility System

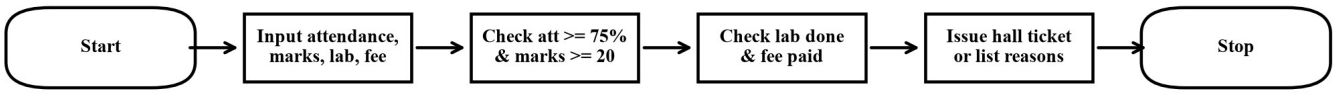
student_name = input("Student name: ")
attendance_percentage = float(input("Attendance percentage (%): "))
internal_marks = float(input("Internal marks (out of 50): "))
is_lab_record_completed = input("Is lab record completed? (yes/no):
").strip().lower()
is_college_fee_cleared = input("Are college fees cleared? (yes/no):
").strip().lower()

lab_ok = (is_lab_record_completed == "yes" or is_lab_record_completed == "y")
fee_ok = (is_college_fee_cleared == "yes" or is_college_fee_cleared == "y")

print("\n--- Exam Eligibility Result ---")
print("Student:", student_name)

if attendance_percentage >= 75.0 and internal_marks >= 20.0 and lab_ok and
fee_ok:
    print("Status: ELIGIBLE FOR EXAMINATIONS (Hall ticket issued)")
else:
    print("Status: NOT ELIGIBLE (Hall ticket withheld)")
    print("Reasons for ineligibility:")
    if attendance_percentage < 75.0:
        print(" - Attendance is below 75%")
    if internal_marks < 20.0:
        print(" - Internal marks below passing score (20/50)")
    if not lab_ok:
        print(" - Lab record incomplete")
    if not fee_ok:
        print(" - College fees pending")
```

Flowchart :



Output :

```
Student name: Yashwanth
Attendance: 82.5% | Internal marks: 38/50
Lab record: yes | Dues cleared: yes
ELIGIBILITY STATUS: ELIGIBLE FOR EXAMINATIONS
Result: Hall ticket is issued to Yashwanth
```

Conclusion: The examination eligibility system automates academic evaluation through multi-stage conditional verification.